



CSC220 · WEB DEVELOPMENT II · WEEK 2 · SESSION 1

Asynchronous JavaScript

Callbacks · Promises

Stamford International University — Information Technology / Computer Science

Lecturers: Thinzar Aung Win · Zakariyya Bature

Today's agenda

10'

Week 1 recap & setup check

ES6+ syntax refresher; everyone can run a .js file with Node

25'

Why asynchronous?

Single-threaded JS, blocking vs non-blocking, timers

25'

Callbacks

Callback pattern, error-first style, callback hell

30'

Promises

States, then/catch/finally, chaining, Promise.all

15'

Class activity

Build a promise chain — 10 points, submit in class

JavaScript runs one thing at a time

- JavaScript is single-threaded: one call stack, one statement at a time
- Some operations are slow: network requests, file access, timers
- Blocking: if we waited synchronously, the whole page would freeze
- Async model: start the slow task, keep going, run a function later when the result arrives

Restaurant analogy: a waiter doesn't stand at one table until the food is cooked — they take the order, serve others, and come back when it's ready.

```
async-intro.js

console.log("1 - order food");

setTimeout(() => {
  console.log("2 - food arrives");
}, 2000);

console.log("3 - keep chatting");

// Output order:
// 1 - order food
// 3 - keep chatting
// 2 - food arrives (after 2s)
```

A function to call when you're done

```
callbacks.js

// a callback is just a function passed
// as an argument, called later
function downloadFile(name, onDone) {
  console.log(`downloading ${name}...`);
  setTimeout(() => {
    onDone(`${name} saved`);
  }, 1500);
}

downloadFile("report.pdf", result => {
  console.log(result);
});

// downloading report.pdf...
// report.pdf saved    (1.5s later)
```

- You already use callbacks: `addEventListener("click", cb)` and `array.map(cb)`
- For async work: “do this task, then call my function with the result”
- Node.js style is error-first: `(err, data) => { ... }` — you'll see it in Week 4
- Works fine for one step — the problem appears when steps depend on each other

Callback hell: the pyramid of doom

```
pyramid.js

login(user, token => {
  getProfile(token, profile => {
    getPosts(profile.id, posts => {
      getComments(posts[0], comments => {
        render(comments, () => {
          console.log("finally done!");
          // ...and where does error
          // handling go? everywhere.
        });
      });
    });
  });
});
```

- Each dependent step nests one level deeper — code grows sideways
- Error handling must be repeated at every level
- Hard to read, hard to reorder, hard to run steps in parallel
- This is the exact problem promises were designed to solve

Coming up: the same 5 steps become a flat, readable chain with promises — and near-synchronous-looking code with `async/await` next session.

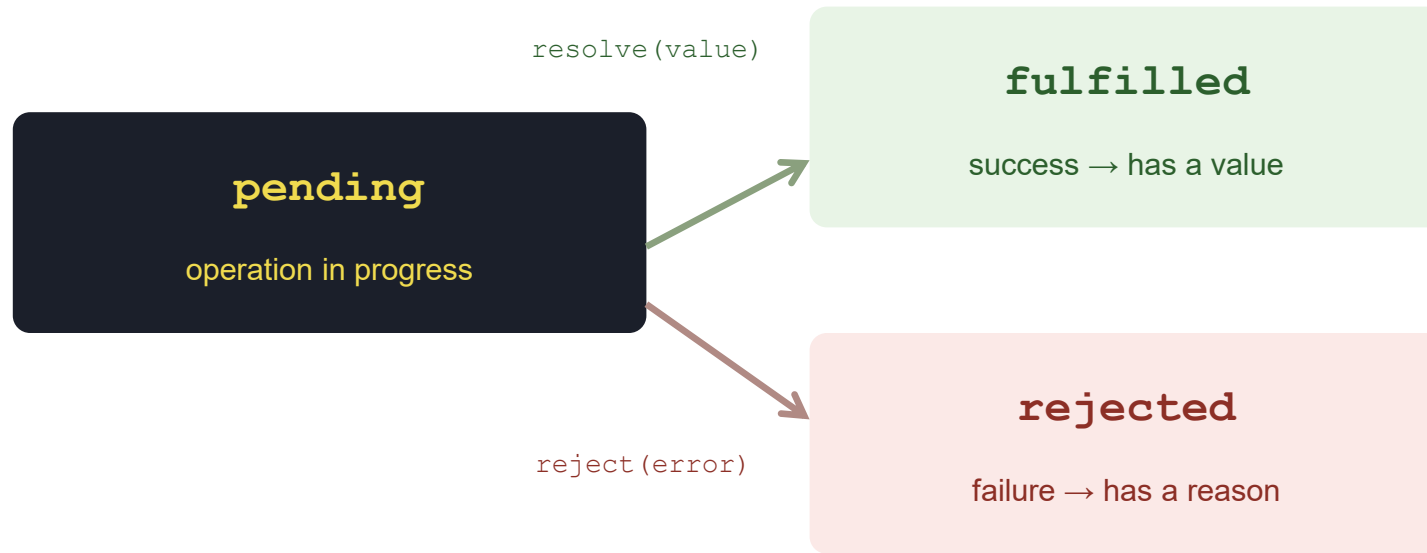
PART 2

Promises

An object that represents a future value —
pending now, fulfilled or rejected later.

.then

One promise, three states



```
promise.js

const order = new Promise(
  (resolve, reject) => {
    setTimeout(() => {
      const ok = Math.random() > 0.3;
      if (ok) resolve("noodles ready");
      else reject(new Error("closed"));
    }, 1000);
  });
```

Settled once, forever. A promise moves from pending to exactly one final state and keeps its value — attaching `.then` later still works.

Consuming: then, catch, finally

```
consume.js

order
  .then(value => {
    console.log("OK:", value);
  })
  .catch(err => {
    console.log("Failed:", err.message);
  })
  .finally(() => {
    console.log("done either way");
  });
```

.then(fn)

runs on success; receives the resolved value

.catch(fn)

runs on failure; receives the error

.finally(fn)

always runs — hide spinners, cleanup

One .catch at the end handles an error from any earlier step — the fix for repeating error handling at every callback level.

Chaining & running in parallel

chain.js

```
// each .then returns a NEW promise:  
// return a value -> passed to next .then  
login(user)  
  .then(token => getProfile(token))  
  .then(profile => getPosts(profile.id))  
  .then(posts => render(posts))  
  .catch(err => showError(err));  
// flat! compare with the pyramid
```

parallel.js

```
// independent tasks? run them together  
Promise.all([  
  getUser(1), getUser(2), getUser(3)  
]).then(([a, b, c]) => {  
  console.log(a.name, b.name, c.name);  
}); // fails fast if ANY rejects
```

- Chain when steps depend on the previous result — sequential
- Promise.all when tasks are independent — parallel, resolves with an array (destructure it!)
- Forgetting to return inside .then is the #1 chaining bug — the next step gets undefined

Activity 3: promise chain (15 min · 10 points)

```
starter.js — provided on LMS

// starter - a reusable delay helper
const delay = (ms, value) =>
  new Promise(resolve =>
    setTimeout(() => resolve(value), ms));

// example
delay(1000, "hello")
  .then(v => console.log(v)); // 1s: hello
```

Your tasks

- 1. Use delay to build a traffic light: log “red”, then “green” after 2 s, then “yellow” after 1 s more — as a flat chain (no nesting)
- 2. Make step 2 reject if a passed value is “broken”; handle it with a single .catch
- 3. Use Promise.all to start three delays together and log all results at once

Submission: upload your .js file to Blackbord “Week 2 – Activity 3” before the end of class. **10 points · during class time only · no late submission.**

What you can do now

- 1 Explain why JavaScript needs asynchronous code
- 2 Use callbacks for “run this later” — and spot callback hell
- 3 Describe the three promise states and how a promise settles
- 4 Consume promises with `then` / `catch` / `finally`
- 5 Chain dependent steps flat, and parallelize with `Promise.all`

Next session

`async/await` — promises with cleaner syntax — and the `Fetch` API: pulling live JSON from public APIs into your pages.